DAY5:PROCEDURAL BLOCKS

Verilog Initial Block:

In Verilog, an **initial block** is a procedural construct that executes exactly once, starting at the very beginning of the simulation (time 0). Unlike always blocks which continuously loop throughout the simulation, an initial block permanently finishes as soon as its last statement completes.

Syntax and Execution Rules:
If an initial block contains only a single statement, it does not need grouping; however, if you have multiple statements, they must be enclosed within begin and end keywords.

- **Sequential Execution:** Inside a standard begin-end initial block, statements execute sequentially. You can introduce timing using delay operators (such as #10) to pause execution for a specific number of time units before moving to the next statement.
- **Parallel Blocks:** You can define any number of initial blocks within a single module. All of them will trigger simultaneously at time 0 and execute independently in parallel.

Purpose and Synthesizability Crucially, initial blocks are **strictly non-synthesizable**. They are completely ignored by logic synthesis tools and cannot be converted into physical hardware logic gates. Therefore, they should never be used in your RTL (Register-Transfer Level) design modules to initialize hardware registers; standard reset signals must be used for actual hardware initialization.
Because of this limitation, initial blocks are exclusively used in **testbenches** for simulation purposes, such as:

- **Initializing signals and memories:** Setting variables to known states right at time 0.
- **Generating stimulus:** Applying sequential test vectors to the Design Under Test (DUT) using delays.
- **Simulation control:** Using system tasks like $finish to define timeouts and explicitly end the simulation run.

```
module tb;
    reg [1:0] a, b;
    initial begin
        a = 2'b10;              // Time 0: a assigned immediately
        #10 b = 2'b00;          // Time 10: b assigned after 10-unit
delay
        #20 a = 2'b01;          // Time 30: a reassigned after
additional 20 units
```

```
        #5 $display("Time=%0t a=%b b=%b", $time, a, b); // Time 35: display values
    end
endmodule
```

- Time 0: a = 2'b10
- Time 10: b = 2'b00
- Time 30: a = 2'b01
- Time 35: Display shows Time=35 a=01 b=00

Common Beginner Mistakes

- **Missing delays in stimulus:** Writing multiple assignments back-to-back without delay operators causes them to all execute simultaneously at time 0, which can create unpredictable race conditions where the exact final value is unknown.
- **Forgetting $finish:** If your testbench includes a continuously running clock (using an always block), the simulation will run indefinitely. You must include an initial block with a delayed $finish statement to explicitly terminate the simulation once your test completes.
- **Using Non-blocking Assignments:** It is a best practice to use blocking assignments (=) rather than non-blocking assignments (<=) inside initial blocks. Mixing them can create timing ambiguities, whereas blocking assignments ensure clear, sequential execution of your test stimulus.

Verilog Always Block:

In Verilog, an **always block** is a procedural construct where statements execute sequentially. Unlike initial blocks that run only once at the start of a simulation, an always block repeats continuously in a loop for the entire duration of the simulation. It is widely used to model concurrently operating hardware, including both combinational logic gates and sequential memory elements.

The Sensitivity List: The execution of an always block is dictated by its **sensitivity list**, which is specified inside parentheses after the @ operator. This list defines the specific events or signal changes that will trigger the block's internal statements to execute.

If an always block completely lacks a sensitivity list and has no internal time delays (like #10), the simulator will hang in a zero-delay infinite loop. For testbenches, you can use explicit delays to create clock generation loops (e.g., always #10 clk = ~clk;), but for actual synthesizable hardware, a sensitivity list is strictly required.

```
// Level-sensitive: Execute always block whenever value of "a" or "b" change
// This is used for combinational logic
always @ (a or b) begin
        [statements]
end
```

Two primary ways to design hardware using always blocks:

1. Modeling Combinational Logic:
To model pure combinational logic (such as ALUs, decoders, or multiplexers), you use a **level-sensitive** sensitivity list. This means the block executes whenever any of its input signals change state.

- **Best Practice:** You should use always @(*) to automatically include all inputs. If you manually list inputs and forget one, the simulation will only trigger when the listed signals change, but the synthesis tool will build hardware based on all inputs. This creates a dangerous mismatch between your simulation and the actual silicon.
- **Assignments:** Inside combinational blocks, you should use **blocking assignments (=)**.
- **Avoid Latches:** You must explicitly define all possible conditions for your outputs (such as providing an else branch for every if). If you fail to do so, Verilog assumes the signal should hold its previous value, and the synthesis tool will infer an unintended hardware latch, which often acts as a bug.

2. Modeling Sequential Logic To model memory elements (such as flip-flops, registers, or counters), you use an **edge-sensitive** sensitivity list.

- **Syntax:** The block is triggered only by specific clock or reset transitions using the posedge (rising edge) or negedge (falling edge) keywords. For example, always @(posedge clk or negedge rstn) is frequently used in ASIC and FPGA designs to create a D flip-flop with an asynchronous reset.
- **Assignments:** You must strictly use **nonblocking assignments (<=)**. If you use blocking assignments (=) inside clocked always blocks, the statements will execute immediately in zero time, which creates unintended combinational paths, ruins pipelined behavior, and causes severe timing violations.

Strict Rules for always Blocks

- **Output Declarations:** Any output signal driven from within a procedural always block must be declared as a **reg** data type. The term reg can be misleading; it does not automatically mean a hardware flip-flop is created. It simply indicates a procedural assignment. It will synthesize into either combinational gates or true flip-flops depending entirely on your sensitivity list.
- **Never Mix Logic Types:** You must never mix edge-sensitive (clock) and level-sensitive (combinational) signals in the same sensitivity list (e.g., always @(posedge clk or a or b)). A block cannot be both combinational and sequential at the same time; this is nonsensical in hardware and will trigger a synthesis error. You must split sequential and combinational logic into separate always blocks.

SENSITIVITY LIST QUICK REFERENCE

Syntax	Trigger type	Use for	Assignment
`always @(*)`	Level-sensitive	Combinational logic (ALU, mux, decoder)	Blocking =
`always @(a or b)`	Level-sensitive	Same — but risky if inputs are forgotten	Blocking =
`always @(posedge clk)`	Edge-sensitive	Flip-flop / register (sync reset)	Non-blocking <=
`always @(posedge clk or negedge rstn)`	Edge-sensitive	Flip-flop with *async* reset	Non-blocking <=
`always #10 clk = ~clk;`	Delay-based	Clock generation in testbenches only	Blocking =

BLOCK COMPARISON

Property	`initial` block	`always` block
Execution	Runs exactly *once*, starts at time 0	Runs *continuously*, loops forever
Trigger	Automatically at simulation start	Sensitivity list (@) or delay
Synthesizable?	No — ignored by tools	Yes (with rules)
Primary use	Testbenches: stimulus, initialization, $finish	RTL design: combinational & sequential logic
Reset in HW?	Never — use reset signal	Use `posedge clk or negedge rstn`
Assignment type	Blocking (=) recommended	Combinational: blocking (=) Sequential: non-blocking (<=)
Multiple blocks?	Yes — all run in parallel at time 0	Yes — each responds to its own sensitivity list
Infinite run risk	No (terminates naturally)	Yes — missing sensitivity list + no delay → hang

Verilog Control Block:

In Verilog, **control blocks** are mechanisms used to implement conditional statements and control the flow of logic, which is essential for accurately modeling digital hardware behavior.
Because these control flow statements dictate how and when data is routed, they must be placed inside procedural blocks (such as always or initial blocks).

The primary control block structures in Verilog:

1. Conditional Statements

- **if-else-if:** This conditional statement evaluates an expression to decide whether a specific block of code should execute, operating very similarly to if-else blocks in the C programming language. The final else part is optional, but it serves as a default "none-of-the-above" case when no other conditions are satisfied.

```
// if statment with an else part
if (expression)
       [statement]
else
       [statement]

// if else for multiple statements should be
// enclosed within "begin" and "end"
if (expression) begin
       [multiple statements]
end else begin
       [multiple statements]
end

// if-else-if statement
if (expression)
       [statement]
else if (expression)
       [statement]
else
       [statement]
```

- **case, casex, and casez:** When a design requires evaluating many different alternatives (like a multiplexer or state machine), a nested if-else-if sequence can become messy and unwieldy. The case statement provides a cleaner shortcut for multiway branching. Verilog also offers variations like casez and casex, which allow the compiler to treat z (high-impedance) and x (unknown) bits as "don't cares" during the comparison process .

2. Looping Statements Verilog provides four different types of loops to execute a block of statements one or more times:

- **for loop:** Controls execution using a three-step process: initializing a loop counter variable, evaluating a condition, and incrementing the counter.
- **while loop:** Continuously executes its internal statements until its evaluated expression evaluates to false.

- **repeat loop:** Executes the loop a strictly fixed number of times.

```
module my_design;
        initial begin
                repeat(4) begin
                        $display("This is a new iteration ...");
                end
        end
endmodule
```

- **forever loop:** Executes infinitely until the simulation is explicitly terminated (typically by a $finish task). It is primarily used in conjunction with timing controls, such as generating a continuous clock signal in a testbench.

3. Common Beginner Mistakes to Avoid

- **Inferring Unintended Latches:** When writing combinational logic inside an always @(*) block, failing to provide an else branch for every if condition (or missing a default case in a case statement) causes Verilog to assume the signal should simply "hold its previous value". Because pure combinational logic cannot store values, the synthesis tool will be forced to build a physical hardware latch to memorize the state, which frequently causes severe timing nightmares. As a best practice, always provide a default else or assign a default value at the top of your block.
- **Zero-Delay Infinite Loops:** When using forever or while loops in simulation testbenches, you must ensure they contain a time delay operator (such as #5). Because these loops execute in zero simulation time by default, running them without a delay will cause the simulator to get stuck in an infinite loop at the current time step and completely hang.

for Synthesizable

Init, condition, increment. Best for iterating over fixed ranges like bus widths.

```
for (i = 0; i < 8; i = i+1)
  data[i] = 0;
```

while Sim only

Loops until condition is false. Needs a delay inside or it hangs.

```
while (count < 10) begin
  #5 count = count + 1;
end
```

⚠ Must have a time delay inside

repeat Sim only

Executes a fixed number of times. No condition — count is set at start.

```
repeat(4) begin
  $display("tick");
  #10;
end
```

forever Testbench

Runs until $finish. Classic use: clock generation. Never use without a delay.

```
initial clk = 0;
always #5 clk = ~clk;
// or:
initial forever
  #5 clk = ~clk;
```

⚠ Must have #delay or simulation hangs

Verilog Block Statements:

In Verilog, **block statements** are used to group a set of statements together so that they are syntactically equivalent to a single statement. They are essential for enabling structured control flow, scoped variable declarations, and modeling concurrent execution.

Two primary types of block statements:

1. Sequential Blocks (begin-end) Statements enclosed between begin and end keywords execute sequentially in the exact order they are written.

- **Delay Semantics:** Delays inside a sequential block are **relative (or cumulative)**. This means a delay operator (like #10) pauses execution relative to the completion time of the *previous* statement.
- **Synthesizability:** Sequential blocks are fully synthesizable and are widely used in Register-Transfer Level (RTL) design to describe hardware behavior.

```
module design0;
	bit [31:0] data;		// SystemVerilog 2-state data type
	// "initial" block starts at time 0
	initial begin
		// Statement 1: Executes at time 0 + 10 = 10 time units
		#10 data = 8'hfe;		// Wait 10 time units, then assign 0xfe
		$display ("[Time=%0t] data=0x%0h", $time, data);
		// Statement 2: Executes at time 10 + 20 = 30 time units (relative delay)
		#20 data = 8'h11;		// Wait 20 MORE time units (cumulative: 30 total)
		$display ("[Time=%0t] data=0x%0h", $time, data);
	end
endmodule
```

2. Parallel Blocks (fork-join) Statements enclosed between fork and join keywords launch simultaneously and execute concurrently as independent threads.

- **Delay Semantics:** Delays inside a parallel block are **absolute**. They are measured directly from the time the fork statement was launched, rather than accumulating after each statement.

- **Completion:** The join keyword blocks further execution of the module until **all** threads within the block have finished executing.
- **Synthesizability:** Parallel blocks are strictly simulation-only constructs. Attempting to use fork-join in synthesizable RTL code will cause an error because synthesis tools cannot convert them into hardware logic. They are primarily used in testbenches to run concurrent monitors, watchdog timers, or parallel transactions.

```
// Basic fork-join block (waits for ALL threads to complete)
fork
       statement1;     // Thread 1: executes concurrently with other
threads
       statement2;     // Thread 2: executes in parallel with thread
1
       statement3;     // Thread 3: executes in parallel with
threads 1 and 2
join                // Blocks until ALL threads finish

// Named fork-join block (for hierarchical access and disable)
fork : block_name
       [statements]    // All statements launch simultaneously at
fork time
join                // Resumes after all threads complete
```

Naming Blocks Both sequential and parallel blocks can be named by adding : name_of_block immediately after the begin or fork keyword. Naming a block provides three major benefits:

1. **Hierarchical References:** It allows you to externally access signals or variables defined inside the block using a hierarchical path (e.g., module.block_name.signal).
2. **Disable Capability:** It allows you to prematurely terminate the block's execution from elsewhere in the code using the disable block_name; statement.
3. **Local Variable Scoping:** It allows you to declare variables that are completely local to that block. Note that in the Verilog-2001 standard, a block **must** be named if you want to declare local variables inside it; unnamed blocks with variable declarations will trigger a compiler error.

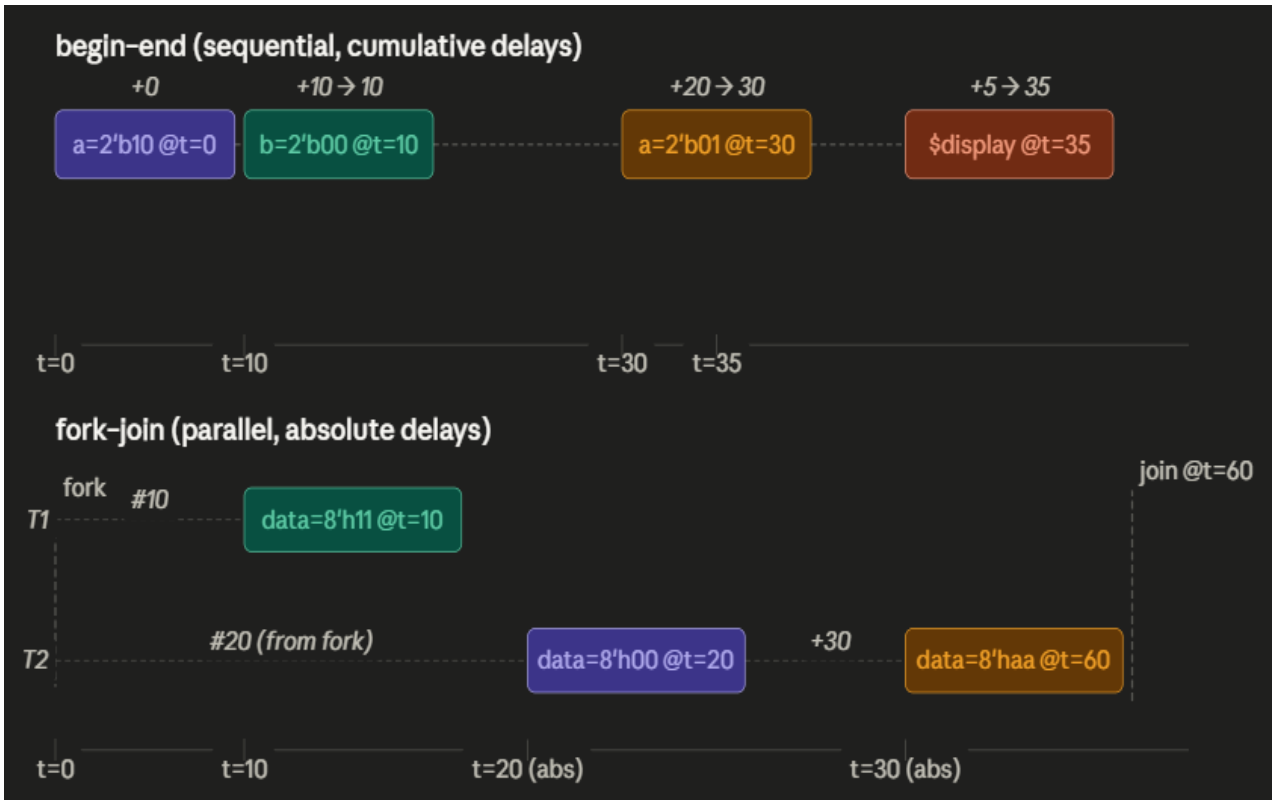
Nested Blocks You can nest blocks within one another to combine their execution rules. For example, placing a begin-end block inside a fork-join block will launch the sequential block as a parallel thread alongside the others, but the statements inside that specific nested block will execute sequentially with cumulative delays.

```
initial begin
```

```
        #10 data = 8'hfe;          // Time 0+10 = 10 (sequential
statement before fork)


        fork                       // Fork launches at time 10
               // Thread 1: Single assignment with delay
               #10 data = 8'h11;       // Time 10+10 = 20 (absolute
delay from fork)


               // Thread 2: Sequential begin-end block (launched in
parallel with Thread 1)
               begin
                      #20 data = 8'h00;     // Time 10+20 = 30 (delay
relative to begin block start)
                      #30 data = 8'haa;     // Time 30+30 = 60
(cumulative delay within begin-end)
               end
        join                       // Resumes at time 60 (when longest
thread completes)
end
```



✓ DO

- ✓ Use `always @(*)` for all combinational logic — never list inputs manually
- ✓ Use blocking `=` inside combinational `always` and `initial` blocks
- ✓ Use non-blocking `<=` inside every clocked `always` block
- ✓ Include `else` or a default assignment to prevent latches
- ✓ Add `$finish` in testbenches with a clock
- ✓ Add time delays in `forever` / `while` loops
- ✓ Declare outputs driven from `always` as `reg`
- ✓ Use `posedge` / `negedge` for flip-flop sensitivity

✗ DON'T

- ✗ Use `initial` blocks to initialize hardware registers — use reset signals
- ✗ Use `non-blocking <=` in initial or combinational blocks — causes ambiguity
- ✗ Mix edge-sensitive and level-sensitive signals in one sensitivity list
- ✗ Use `fork-join` in synthesizable RTL — simulation only
- ✗ Use `forever` or `while` without a `#delay`
- ✗ Skip the `default` case in a `case` statement — infers latch
- ✗ Write multiple stimulus assignments at the same time — creates race conditions
- ✗ Use an `always` block without a sensitivity list and no delay — sim hangs

ASSIGNMENT RULES MEMORY AID

initial block	always @(*)	always @(posedge)
= blocking	= blocking	<= non-blocking
clear, sequential stimulus	combinational gates, no memory	registers, flip-flops, pipelines

VERILOG PROCEDURAL BLOCKS, CONTROL AND BLOCK STATEMENTS

1. PROCEDURAL BLOCKS

Procedural blocks execute statements sequentially.

Main procedural blocks:

- initial
- always

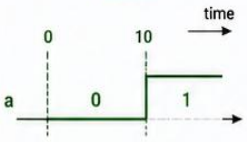
Feature	initial	always
Execution count	Once	Infinite
Starts at	Time 0	Time 0
Main use	Testbench / Initialization	Hardware modeling
Synthesizable?	Usually no	Yes

1.1 initial Block

Executes only once from time 0.

```
module test;
  reg a;
  initial
  begin
    a = 0;        // at time 0
    #10 a = 1;    // after 10 time units
  end
endmodule
```

Behavior



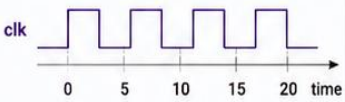
- time 0 → a = 0
- after 10 units → a = 1
- block finishes permanently

1.2 always Block

Executes repeatedly forever.

```
module clk_gen;
  reg clk;
  initial clk = 0;
  always #5 clk = ~clk;
endmodule
```

Clock Waveform

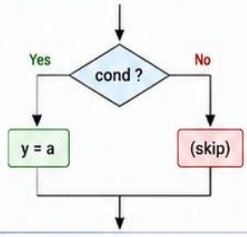


- Clock toggles every 5 time units forever.

2. CONTROL STATEMENTS

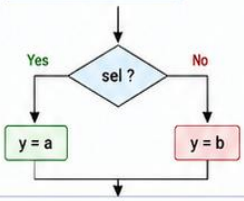
2.1 if Statement

```
if (cond)
  y = a;
```



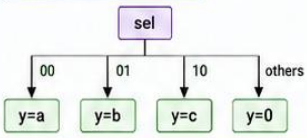
2.2 if-else Statement

```
if (sel)
  y = a;
else
  y = b;
```



2.3 case Statement

```
case (sel)
  2'b00 : y = a;
  2'b01 : y = b;
  2'b10 : y = c;
 default: y = 0;
endcase
```



2.4 Loops

```
integer i;
initial begin
  for (i = 0; i < 4; i = i+1)
    $display("i = %0d", i);
end
```

- for loop
- while loop
- repeat loop
- forever loop

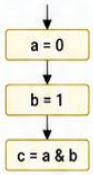
3. BLOCK STATEMENTS

3.1 Sequential Block

begin - end

```
begin
  a = 0;
  b = 1;
  c = a & b;
end
```

Statements execute one after another

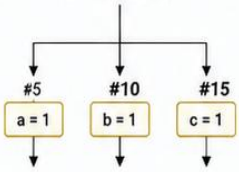


3.2 Parallel Block

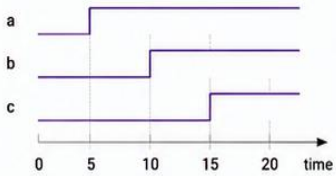
fork - join

```
fork
  #5  a = 1;
  #10 b = 1;
  #15 c = 1;
join
```

Statements execute **simultaneously**



Example Waveform



4. TIMING CONTROL STATEMENTS

4.1 Delay Control (#)

Executes after specified delay.

```
initial begin
  a = 0;
  #10 a = 1;    // after 10 time units
end
```

4.2 Event Control (@)

Waits for an event.

```
always @(posedge clk)
  q <= d;
always @(negedge rst_n)
  q <= 0;
```

QUICK SUMMARY

- initial → executes once at time 0.
- always → executes forever.
- **if / if-else** / case → control flow.
- **begin-end** → sequential execution.
- **fork-join** → parallel execution.
- # → delay control.
- @ → event control.

5. COMMON PROCEDURAL MODELING STYLES

Style	Syntax	Use
Combinational Logic	always @(*)	For combinational circuits
Sequential Logic	always @(posedge clk)	For flip-flops, registers
Async Reset	always @(posedge clk or posedge rst) // or negedge rst_n	For asynchronous reset

Sync Reset	always @(posedge clk)
	if (!rst_n)
	q <= 0;
	else
	q <= d;
		For synchronous reset